Unidad 8 · SQL para crear y modificar: DDL y DML

Qué vas a poder hacer. Escribir el `CREATE TABLE` de un diseño relacional eligiendo tipos y restricciones que impidan guardar datos malos, cargar filas con `INSERT`, cambiar la estructura con `ALTER TABLE`, corregir y borrar datos con `UPDATE` y `DELETE` sin tocar más filas de las que corresponden, y leer el mensaje de error de la base para saber qué restricción saltó y por qué.

Lo que necesitas antes. De la unidad 5: clave primaria, clave foránea e integridad referencial. De la unidad 6: por qué cada hecho se guarda en una sola tabla. Haber abierto `sqlite3` (viene con Python) o MySQL Workbench una vez.

Tiempo. Unas 5 horas: 2 de lectura con `sqlite3` abierto, corriendo cada bloque, y 3 de práctica.

---

1. La idea: dos lenguajes dentro de SQL

SQL tiene dos partes que se usan distinto:

- **DDL** (*data definition language*): `CREATE TABLE`, `ALTER TABLE`, `DROP TABLE`. Define la **estructura**: qué tablas hay, qué columnas, de qué tipo y con qué reglas. Se escribe una vez y se cambia poco.
- **DML** (*data manipulation language*): `INSERT`, `UPDATE`, `DELETE` (y `SELECT`, que se ve desde la unidad 9). Cambia el **contenido**, fila por fila, todos los días.

La estructura es un contrato. Cada `NOT NULL`, `UNIQUE`, `CHECK` o `FOREIGN KEY` es una regla que la base hace cumplir sola, para siempre, sin importar qué programa o qué persona inserte los datos. Una planilla no puede hacer eso: acepta cualquier cosa en cualquier celda. Por eso el diseño de las restricciones es parte del diseño de la base, no un detalle de implementación: una regla que no está en el `CREATE TABLE` es una regla que alguien va a romper.

Un script se lee de arriba abajo. Una tabla que tiene una clave foránea solo se puede crear después de la tabla a la que apunta, y sus filas solo se pueden cargar después de las filas de esa tabla. Para borrar es al revés: primero los hijos, después los padres.

---

2. `CREATE TABLE` y los tipos de datos

```
CREATE TABLE nombre_tabla (
    columna1  TIPO  [restricciones de la columna],
    columna2  TIPO  [restricciones de la columna],
    ...
    [restricciones de la tabla: claves compuestas, claves foráneas, CHECK entre columnas]
);
```

Los tipos que usa este ramo:

Para guardar	SQLite	MySQL	Por qué
un entero (cantidades, códigos)	`INTEGER`	`INT`	
pesos chilenos	`INTEGER`	`INT`	el peso no tiene decimales; nunca `FLOAT` para plata (redondea)
un monto o medida con decimales	`DECIMAL(p,s)` (se guarda como `REAL`)	`DECIMAL(p,s)` (exacto)	`DECIMAL(5,1)` = hasta 9.999,9
un texto corto	`VARCHAR(n)`	`VARCHAR(n)`	SQLite no corta en `n`; MySQL sí lo exige
un texto largo	`TEXT`	`TEXT`	observaciones, descripciones
una fecha	`DATE` (texto `'AAAA-MM-DD'`)	`DATE`	solo el formato año-mes-día ordena bien
sí o no	`INTEGER` con 0 y 1	`BOOLEAN` (es `TINYINT(1)`)	SQLite no tiene tipo lógico
un teléfono, un RUT	`VARCHAR`	`VARCHAR`	no son números: no se suman, llevan `+`, `-` o `K`

SQLite es flexible con los tipos y MySQL no. SQLite asigna a cada columna una *afinidad* según el nombre del tipo: si contiene `INT` es entera; si contiene `CHAR`, `CLOB` o `TEXT` es de texto; y así. El número entre paréntesis de `VARCHAR(40)` lo ignora: guarda un texto de 50 letras sin reclamar (ejercicio 8.2). `DATE` no es un tipo propio: la fecha se guarda como texto, y las funciones de fecha solo la entienden en formato `'AAAA-MM-DD'`. MySQL, con el modo estricto que trae activado por defecto, rechaza un texto más largo que su `VARCHAR` y una fecha que no sea fecha. Si quieres que SQLite también controle los tipos, agrega la palabra `STRICT` después del paréntesis final del `CREATE TABLE` (desde SQLite 3.37); este ramo no la usa para que los scripts corran igual en MySQL.

Los números entre paréntesis de los enteros de MySQL (`INT(9)`) no limitan nada. Son un «ancho de despliegue» antiguo que MySQL 8 marca como obsoleto. Un `INT` guarda hasta 2.147.483.647, con o sin paréntesis.

3. Restricciones: lo que la base se niega a guardar

Restricción	Qué impide	Dónde se escribe
`PRIMARY KEY`	dos filas con la misma clave	en la columna, o al final si es compuesta: `PRIMARY KEY (a, b)`
`NOT NULL`	dejar la columna vacía	en la columna
`UNIQUE`	repetir un valor (o una combinación: `UNIQUE (a, b)`)	en la columna o al final
`CHECK (condición)`	una fila que no cumple la condición	en la columna, o al final si usa dos columnas
`DEFAULT valor`	nada: pone ese valor si el `INSERT` no menciona la columna	en la columna
`FOREIGN KEY (col) REFERENCES t(col)`	apuntar a una fila que no existe, y borrar una fila a la que alguien apunta	al final

`NOT NULL` y `UNIQUE` son independientes. `UNIQUE` permite varios `NULL` (un `NULL` no es igual a otro `NULL`); si la columna tiene que estar siempre llena y sin repetirse, lleva las dos, como el RUT del ejemplo.

Qué pasa al borrar un padre. La clave foránea decide qué hacer con los hijos cuando se borra la fila a la que apuntan:

Acción	Qué hace	Cuándo
sin decir nada (`NO ACTION` o `RESTRICT`)	rechaza el borrado mientras haya hijos	casi siempre: obliga a decidir qué hacer con los hijos
`ON DELETE CASCADE`	borra también a los hijos	los hijos no tienen sentido solos (las líneas de un pedido)
`ON DELETE SET NULL`	deja la clave foránea de los hijos en `NULL`	el hijo sigue existiendo sin padre (un alumno sin taller)

Dos trampas que hay que conocer.

- **SQLite no revisa las claves foráneas si no se lo pides.** Hay que ejecutar `PRAGMA foreign_keys = ON;` en **cada conexión**, antes de cargar datos. Sin eso, la cláusula `FOREIGN KEY` queda escrita pero no hace nada (el caso de la sección 14 lo muestra).
- **MySQL 8.0 y 8.4 ignoran una clave foránea escrita al lado de la columna** (`id_tienda INT REFERENCES tienda(id_tienda)`): la aceptan sin error y no crean la restricción. Recién MySQL 9 la acepta. La forma que funciona en todas las versiones es la línea aparte `FOREIGN KEY (id_tienda) REFERENCES tienda(id_tienda)`, que es la que usan los scripts de esta unidad. Y MySQL hace cumplir los `CHECK` desde la versión 8.0.16; antes los aceptaba y los ignoraba.

SQLite y la clave primaria con `NULL`. Por un error antiguo que conserva por compatibilidad, SQLite deja guardar `NULL` en una columna `PRIMARY KEY` que no sea `INTEGER PRIMARY KEY`. Por eso en una clave compuesta conviene escribir `NOT NULL` en cada columna, como hace el esquema del caso.

4. Claves que se numeran solas

	SQLite	MySQL
Cómo se declara	`id INTEGER PRIMARY KEY`	`id INT AUTO_INCREMENT PRIMARY KEY`
Qué pasa si el `INSERT` no la trae	usa el mayor valor actual + 1	usa el siguiente valor de un contador
Si se borra la fila con el número mayor	el número **se vuelve a usar**	no se vuelve a usar
Para que nunca se reutilice	`INTEGER PRIMARY KEY AUTOINCREMENT`	(ya es así)

En SQLite tiene que decir exactamente `INTEGER` (no `INT` ni `BIGINT`) para numerarse sola. Reutilizar un número importa cuando el número sale de la base: un ticket, una boleta o un folio que alguien anotó no debería volver a aparecer con otro contenido (ejercicio 8.8).

5. `ALTER TABLE` y `DROP TABLE`

Cambio	SQLite	MySQL
agregar una columna	`ALTER TABLE t ADD COLUMN c tipo;`	igual
cambiar el nombre de una columna	`ALTER TABLE t RENAME COLUMN a TO b;`	igual (MySQL 8), o `CHANGE COLUMN a b tipo`
quitar una columna	`ALTER TABLE t DROP COLUMN c;`	igual
cambiar el nombre de la tabla	`ALTER TABLE t RENAME TO t2;`	`RENAME TABLE t TO t2;` o igual
cambiar el tipo o las restricciones de una columna	**no existe**: se reconstruye la tabla	`ALTER TABLE t MODIFY COLUMN c tipo;`

Una columna nueva con `NOT NULL` necesita un `DEFAULT`: las filas que ya existen tienen que quedar con algún valor. `MODIFY` **no cambia nombres**: cambia el tipo de una columna que conserva su nombre; para cambiar nombre y tipo a la vez, MySQL usa `CHANGE COLUMN viejo nuevo tipo`.

Reconstruir una tabla en SQLite (ejercicio 8.5) son cuatro pasos: crear la tabla nueva con la definición correcta, copiar las filas con `INSERT INTO nueva SELECT ... FROM vieja`, borrar la vieja y renombrar la nueva.

`DROP TABLE t;` borra la tabla **con todos sus datos**, sin papelera. `DROP TABLE IF EXISTS t;` no reclama si la tabla no existe (útil al comienzo de un script que se corre varias veces). Con las claves foráneas activas, no se puede borrar una tabla mientras otra tenga filas que apunten a ella.

6. `INSERT`: cargar filas

```
INSERT INTO tabla VALUES (v1, v2, v3);                      -- todas las columnas, en
orden
INSERT INTO tabla (col1, col3) VALUES (v1, v3);             -- solo esas; el resto,
DEFAULT o NULL
INSERT INTO tabla (col1, col2) VALUES (a1, a2), (b1, b2);   -- varias filas en una
instrucción
INSERT INTO tabla (col1, col2) SELECT x, y FROM otra;       -- las filas que devuelve una
consulta
```

- **Nombra las columnas.** La forma sin lista depende del orden de las columnas en el `CREATE TABLE`: si mañana alguien agrega una, todos los `INSERT` sin lista se rompen o, peor, guardan cada valor en la columna equivocada.
- `DEFAULT` **actúa solo cuando la columna no aparece.** Si el `INSERT` nombra la columna y le pasa `NULL`, queda `NULL` (ejercicio 8.3).
- **Comillas simples para los textos y las fechas:** `'Halcones'`, `'2026-10-03'`. En SQL estándar las comillas dobles son para nombres de columnas; MySQL las acepta como texto, pero un script que las use no es portable.
- **Una instrucción con varias filas es todo o nada:** si una fila viola una restricción, no entra ninguna de esa instrucción.

7. `UPDATE` y `DELETE`: el `WHERE` es la mitad de la instrucción

```
UPDATE tabla SET col1 = valor1, col2 = expresion WHERE condicion;
DELETE FROM tabla WHERE condicion;
```

Sin `WHERE`, la instrucción afecta a todas las filas. `UPDATE producto SET precio = 3390;` le pone ese precio a todo el catálogo, y la base no pregunta: es una instrucción perfectamente válida. Es el error más caro del ramo porque no da mensaje. Cuatro hábitos que lo evitan:

1. **Escribe primero el** `SELECT` con el mismo `WHERE` y mira qué filas trae. Si son las que quieres cambiar, reemplaza `SELECT *` por `UPDATE ... SET` o `DELETE`.
2. **Filtra por la clave primaria** cuando cambias una fila: `WHERE id_partido = 2`, no `WHERE fecha = '2026-10-10'` (que puede traer dos partidos).
3. **Mira cuántas filas cambiaron.** En SQLite, `SELECT changes();` justo después; en MySQL lo informa el mismo cliente («1 row affected»); desde Python, `cursor.rowcount`.
4. **Cambia dentro de una transacción.** `BEGIN;`, el `UPDATE`, se revisa `changes()` y se termina con `COMMIT;` (se guarda) o `ROLLBACK;` (se deshace todo desde el `BEGIN`). La unidad 12 trata las transacciones a fondo.

MySQL trae además un seguro: con `SET sql_safe_updates = 1;` (o el cliente `mysql --safe-updates`) rechaza un `UPDATE` o un `DELETE` cuyo `WHERE` no use una clave y que no tenga `LIMIT`. MySQL Workbench tiene la misma opción («Safe Updates») en sus preferencias del editor SQL.

`DELETE` **no es** `DROP`**.** `DELETE FROM t;` vacía la tabla y la deja creada; `DROP TABLE t;` la hace desaparecer con su estructura.

8. Desde un programa: nunca armar el SQL pegando texto

Cuando un programa en Python carga datos, el valor tiene que viajar **aparte** de la instrucción, con un marcador `?`. Si se pega el texto dentro de la instrucción, un dato con comillas cambia lo que la instrucción hace (eso es la *inyección SQL*):

```
nombre = "x' OR '1'='1"                                    # lo que alguien escribió
en un formulario
con.execute(f"DELETE FROM socio WHERE nombre = '{nombre}'")     # MAL: la condición queda
siempre verdadera
con.execute("DELETE FROM socio WHERE nombre = ?", (nombre,))    # BIEN: busca a alguien
llamado así
```

Sobre la tabla de socios del ejercicio 8.3, la primera línea borra las 4 filas; la segunda borra 0 filas, porque nadie se llama `x' OR '1'='1`. El script del caso carga sus 405 filas así, con `executemany` y marcadores. La unidad 12 vuelve sobre esto.

9. Diferencias entre SQLite y MySQL en esta unidad

Tema	SQLite	MySQL 8
claves foráneas	apagadas hasta `PRAGMA foreign_keys = ON;`	siempre activas (motor InnoDB)
`REFERENCES` al lado de la columna	funciona	se ignora en 8.0 y 8.4; usa `FOREIGN KEY (...)`
`CHECK`	siempre	desde 8.0.16
largo de `VARCHAR(n)` y formato de `DATE`	no los controla	los controla (modo estricto)
clave que se numera sola	`INTEGER PRIMARY KEY`	`AUTO_INCREMENT`
cambiar el tipo de una columna	reconstruir la tabla	`MODIFY COLUMN`
ver filas cambiadas	`SELECT changes();`	lo informa el cliente; `ROW_COUNT()`
seguro contra `UPDATE` sin `WHERE`	no tiene	`sql_safe_updates`

10. Ejemplo resuelto · La liga vecinal de básquetbol

Una liga de básquetbol de Los Ángeles quiere dejar la planilla y registrar equipos, jugadores y partidos en una base. Las reglas que da la directiva:

- cada equipo tiene un nombre que no se repite y una comuna (casi todos son de Los Ángeles);
- cada jugador tiene RUT (no se repite), nombre, fecha de nacimiento y un número de camiseta entre 0 y 99, y juega en un equipo; en un mismo equipo no puede haber dos camisetas iguales;
- cada partido tiene fecha, equipo local y visita (distintos) y el puntaje de cada uno, que queda vacío hasta que se juega.

Todos los bloques están en `sql/u08_liga.sql`.

Paso 1 · Crear las tablas, padres primero

```
PRAGMA foreign_keys = ON;

CREATE TABLE equipo (
    id_equipo  INTEGER PRIMARY KEY,
    nombre     VARCHAR(30) NOT NULL UNIQUE,
    comuna     VARCHAR(30) NOT NULL DEFAULT 'Los Ángeles'
);

CREATE TABLE jugador (
    id_jugador INTEGER PRIMARY KEY,
    rut        VARCHAR(12) NOT NULL UNIQUE,
    nombre     VARCHAR(40) NOT NULL,
    fecha_nac  DATE NOT NULL,
    camiseta   INTEGER NOT NULL CHECK (camiseta BETWEEN 0 AND 99),
    id_equipo  INTEGER NOT NULL,
    UNIQUE (id_equipo, camiseta),
    FOREIGN KEY (id_equipo) REFERENCES equipo(id_equipo)
);

CREATE TABLE partido (
    id_partido INTEGER PRIMARY KEY,
    fecha      DATE NOT NULL,
    id_local   INTEGER NOT NULL,
    id_visita  INTEGER NOT NULL,
    pts_local  INTEGER CHECK (pts_local >= 0),
    pts_visita INTEGER CHECK (pts_visita >= 0),
    CHECK (id_local <> id_visita),
    FOREIGN KEY (id_local)  REFERENCES equipo(id_equipo),
    FOREIGN KEY (id_visita) REFERENCES equipo(id_equipo)
);
```

Cada regla de la directiva quedó en una línea: el RUT con `NOT NULL UNIQUE`, la camiseta con un `CHECK` de rango y un `UNIQUE (id_equipo, camiseta)` compuesto, la comuna con `DEFAULT`, los equipos distintos con un `CHECK` que compara dos columnas (por eso va al final). Los puntajes **no** llevan `NOT NULL`: un partido por jugar no tiene puntaje, y `NULL` dice exactamente eso.

Paso 2 · Cargar los equipos

```
INSERT INTO equipo (id_equipo, nombre, comuna) VALUES
    (1, 'Halcones', 'Los Ángeles'),
    (2, 'Toros de Mulchén', 'Mulchén'),
    (4, 'Nacimiento BC', 'Nacimiento');
INSERT INTO equipo (nombre) VALUES ('Escuela Industrial');
SELECT * FROM equipo;
```

id_equipo	nombre	comuna
1	Halcones	Los Ángeles
2	Toros de Mulchén	Mulchén
4	Nacimiento BC	Nacimiento
5	Escuela Industrial	Los Ángeles

La Escuela Industrial no trajo ni clave ni comuna: la clave se generó como el mayor valor más uno (5, no 3: no rellena huecos) y la comuna salió del `DEFAULT`.

Paso 3 · Cargar los jugadores

```
INSERT INTO jugador (id_jugador, rut, nombre, fecha_nac, camiseta, id_equipo) VALUES
    (1, '43.538.745-4', 'Tomás Sepúlveda',     '2001-04-17',  7, 1),
    (2, '42.527.675-1', 'Ignacio Riquelme',    '1998-11-02', 23, 1),
    (3, '43.740.959-5', 'Benjamín Toro',       '2003-01-29',  7, 2),
    (4, '42.932.103-4', 'Vicente Jara',        '1999-08-12', 11, 2),
    (5, '43.437.638-6', 'Felipe Castro',       '2002-06-05',  4, 5),
    (6, '43.235.424-5', 'Cristóbal Navarrete', '2000-03-21', 15, 4);
SELECT COUNT(*) FROM jugador;
```

COUNT(*)
6

Dos jugadores usan la camiseta 7, pero en equipos distintos: el `UNIQUE (id_equipo, camiseta)` lo permite.

Paso 4 · Seis filas que la base rechaza

Cada `INSERT` de abajo rompe una regla distinta. Lo que importa es leer el mensaje: dice qué restricción saltó y en qué columna.

```
INSERT INTO jugador (rut, fecha_nac, camiseta, id_equipo) VALUES ('43.943.173-3', '2004-09-
30', 9, 1);
```

`NOT NULL constraint failed: jugador.nombre` (falta el nombre).

```
INSERT INTO jugador (rut, nombre, fecha_nac, camiseta, id_equipo)
VALUES ('42.527.675-1', 'Martín Riquelme', '2004-09-30', 9, 1);
```

`UNIQUE constraint failed: jugador.rut` (es el RUT de Ignacio: alguien lo copió mal).

```
INSERT INTO jugador (rut, nombre, fecha_nac, camiseta, id_equipo)
VALUES ('43.943.173-3', 'Martín Riquelme', '2004-09-30', 120, 1);
```

`CHECK constraint failed: camiseta BETWEEN 0 AND 99`

```
INSERT INTO jugador (rut, nombre, fecha_nac, camiseta, id_equipo)
VALUES ('43.943.173-3', 'Martín Riquelme', '2004-09-30', 9, 3);
```

`FOREIGN KEY constraint failed` (no hay equipo 3). SQLite no dice cuál clave foránea; con varias en la tabla, hay que revisar cada una.

```
INSERT INTO jugador (rut, nombre, fecha_nac, camiseta, id_equipo)
VALUES ('43.943.173-3', 'Martín Riquelme', '2004-09-30', 7, 1);
```

`UNIQUE constraint failed: jugador.id_equipo, jugador.camiseta` (Tomás ya usa la 7 en Halcones).

```
INSERT INTO partido (fecha, id_local, id_visita) VALUES ('2026-10-10', 2, 2);
```

```
CHECK constraint failed: id_local <> id_visita
```

Ninguna de las seis filas quedó guardada: la tabla sigue con 6 jugadores.

Paso 5 · Programar partidos y anotar un resultado

```
INSERT INTO partido (id_partido, fecha, id_local, id_visita, pts_local, pts_visita) VALUES
    (1, '2026-10-03', 1, 2, 68, 61);
INSERT INTO partido (id_partido, fecha, id_local, id_visita) VALUES
    (2, '2026-10-10', 5, 4),
    (3, '2026-10-10', 2, 1);
SELECT * FROM partido;
```

id_partido	fecha	id_local	id_visita	pts_local	pts_visita
1	2026-10-03	1	2	68	61
2	2026-10-10	5	4	NULL	NULL
3	2026-10-10	2	1	NULL	NULL

El sábado 10 se juega el partido 2 y termina 54 a 59. El `WHERE` va por la clave: por fecha habría cambiado también el partido 3.

```
UPDATE partido
SET pts_local = 54, pts_visita = 59
WHERE id_partido = 2;
SELECT changes();
```

changes()
1

Paso 6 · Agregar una columna

La directiva pide guardar un teléfono de contacto. La columna nueva queda en `NULL` para todos hasta que se llene:

```
ALTER TABLE jugador ADD COLUMN telefono VARCHAR(15);
UPDATE jugador SET telefono = '+56 9 0000 0023' WHERE id_jugador = 2;
SELECT id_jugador, nombre, telefono FROM jugador WHERE id_equipo = 1;
```

id_jugador	nombre	telefono
1	Tomás Sepúlveda	NULL
2	Ignacio Riquelme	+56 9 0000 0023

Paso 7 · Nacimiento BC se retira de la liga

Primero se intenta borrar el equipo:

```
DELETE FROM equipo WHERE id_equipo = 4;
```

`FOREIGN KEY constraint failed`. La base protege a Cristóbal Navarrete y al partido 2, que apuntan a ese equipo. Es lo que se quiere: obliga a decidir qué pasa con ellos. La directiva decide dar de baja al jugador y anular el partido:

```
DELETE FROM jugador WHERE id_jugador = 6;
SELECT changes();
```

changes()
1

```
DELETE FROM partido WHERE id_local = 4 OR id_visita = 4;
DELETE FROM equipo WHERE id_equipo = 4;
SELECT id_equipo, nombre FROM equipo;
```

id_equipo	nombre
5	Escuela Industrial
1	Halcones
2	Toros de Mulchén

(SQLite devuelve las filas en el orden que le resulta más cómodo, aquí el del índice del nombre; sin `ORDER BY` no hay orden garantizado.)

Paso 8 · Cómo quedó

```
SELECT id_jugador, nombre, camiseta, id_equipo, telefono FROM jugador;
```

id_jugador	nombre	camiseta	id_equipo	telefono
1	Tomás Sepúlveda	7	1	NULL
2	Ignacio Riquelme	23	1	+56 9 0000 0023
3	Benjamín Toro	7	2	NULL
4	Vicente Jara	11	2	NULL
5	Felipe Castro	4	5	NULL

Cómo se lee.

- **Las reglas de la directiva ya no dependen de quien digite.** Los seis errores del paso 4 son errores reales de una planilla (un nombre que falta, un RUT copiado, una camiseta 120, un equipo que no existe, un número repetido, un partido de un equipo contra sí mismo); en la base no entran.
- **Un error de la base es información.** El mensaje dice qué regla se rompió y en qué columna; con `FOREIGN KEY constraint failed` hay que buscar qué clave foránea apunta a la nada.

- **Borrar un equipo obligó a una decisión explícita** sobre sus jugadores y partidos. Si la directiva hubiera querido borrar todo de una vez, la clave foránea tendría que haberse declarado con `ON DELETE CASCADE` (ejercicio 8.7); sin decirlo, la base no borra nada por su cuenta.
- **Cada `UPDATE` y cada `DELETE` fue por clave primaria y se revisó con `changes()`**: una fila cada vez, la que se quería.

11. Errores típicos

Error	Cómo evitarlo
`UPDATE` o `DELETE` sin `WHERE`	escribe primero el `SELECT` con el mismo `WHERE`; trabaja dentro de `BEGIN` y revisa `changes()` antes del `COMMIT`
Olvidar `NOT NULL` en lo obligatorio	por cada columna, pregúntate: ¿puede existir la fila sin este dato? Si no, `NOT NULL`
Clave foránea escrita pero no activa (`PRAGMA` en SQLite, `REFERENCES` en línea en MySQL 8)	en SQLite, `PRAGMA foreign_keys = ON;` al abrir; en MySQL, `FOREIGN KEY (...)` en línea aparte; prueba con un `INSERT` que apunte a la nada: tiene que fallar
Crear o cargar un hijo antes que su padre	ordena el script: tablas sin claves foráneas primero; para borrar, al revés
Fechas como `'05/11/2026'`	siempre `'AAAA-MM-DD'`: es la que ordena bien como texto y la que entienden las funciones de fecha
Teléfonos o RUT como número	`VARCHAR`: no se suman, llevan `+`, guiones o `K`, y un número pierde los ceros de la izquierda
Textos sin comillas o con comillas dobles	comillas simples: `'Pedro'`; sin comillas, SQL lo lee como el nombre de una columna
`FOREIGN KEY col REFERENCES ...` sin paréntesis	`FOREIGN KEY (col) REFERENCES tabla(col)`
Usar `MODIFY` para cambiar un nombre	`MODIFY` cambia el tipo; el nombre se cambia con `RENAME COLUMN` (o `CHANGE` en MySQL)
`INSERT` sin lista de columnas	nombra las columnas: el script sigue funcionando si la tabla cambia
Armar el SQL pegando lo que escribió el usuario	marcadores `?` y el valor aparte (sección 8)

12. Práctica

Diez ejercicios: 8.1 a 8.4 básicos, 8.5 a 8.8 intermedios, 8.9 y 8.10 de desafío. Todas las soluciones están en `sql/u08_ejercicios.sql`, con un bloque por paso. Escribe tu versión, córrela en una base nueva y compara los mensajes y las filas con las pautas de la sección 13.

8.1 · La ficha de una mascota (básico, escribir). Un consultorio veterinario de Chillán guarda de cada mascota un código que se numera solo, su nombre (obligatorio), la especie (solo `perro`, `gato` u `otro`), la fecha de nacimiento (si se sabe), el peso en kilos con un decimal (positivo, si se midió) y el RUT del dueño (obligatorio). Escribe el `CREATE TABLE`. Carga a Canela, perra nacida el 14-05-2019 que pesa 12,5 kg, y prueba que la base rechace un loro y un gato de 0 kg. ¿Qué cambia en MySQL?

8.2 · Tipos y formatos (básico). (a) Elige el tipo, en SQLite y en MySQL, para: el precio de una consulta en pesos, el teléfono del dueño, la fecha de la próxima vacuna, el peso, si la mascota está esterilizada y las observaciones del veterinario. (b) En SQLite, crea una tabla con una columna `comuna VARCHAR(5)` e inserta `'Concepción'`. ¿Qué queda guardado? ¿Qué haría MySQL? (c) Guarda las fechas de tres controles como texto `'07/03/2026'`, `'15/01/2026'` y `'28/02/2025'`, y también en formato `'AAAA-MM-DD'`. Ordena cada columna. ¿Cuál orden es el cronológico?

8.3 · Lo que pone `DEFAULT` (básico, predecir). Un club deportivo crea esta tabla y corre cuatro `INSERT`. Sin ejecutar, escribe cómo queda la tabla; después compruébalo.

```
CREATE TABLE socio (
    id_socio  INTEGER PRIMARY KEY,
    nombre    VARCHAR(40) NOT NULL,
    categoria VARCHAR(10) NOT NULL DEFAULT 'adulto',
    cuota     INTEGER DEFAULT 14500,
    activo    INTEGER NOT NULL DEFAULT 1
);
INSERT INTO socio VALUES (1, 'Antonia Bravo', 'juvenil', 9800, 1);
INSERT INTO socio (nombre) VALUES ('Raúl Espinoza');
INSERT INTO socio (nombre, cuota) VALUES ('Gloria Pinto', NULL);
INSERT INTO socio (nombre, categoria, activo) VALUES ('Sergio Vidal', 'senior', 0);
SELECT * FROM socio;
```

8.4 · El orden de un script (básico). Un CESFAM registra los box de atención y las atenciones de cada día (cada atención ocurre en un box). (a) Escribe las dos tablas con su clave foránea. (b) ¿Qué pasa si el script carga la atención 1 (en el box 3) antes de cargar el box 3? (c) Cargados los box 3 y 7 y tres atenciones, ¿se puede borrar la tabla `box`? ¿En qué orden se borran las dos tablas?

8.5 · Cambiar una tabla con datos (intermedio). La feria de Tomé guarda sus puestos en `puesto(id_puesto, titular, metros)`, con tres puestos de 6, 9 y 4 metros cuadrados. (a) Agrega la columna `rubro`, obligatoria, con valor `'verduras'` para los que ya existen. (b) Cambia el nombre de `metros` a `metros2`. (c) El puesto 3 en realidad mide 4,5 m²: hay que cambiar el tipo de `metros2` a `DECIMAL(4,1)` y exigir que sea positivo. Intenta `MODIFY COLUMN` en SQLite, lee el error y haz la reconstrucción en cuatro pasos sin perder datos. (d) Quita la columna `rubro`.

8.6 · Subir precios con una expresión (intermedio). Una librería escolar de Coronel sube 8 % los cuadernos y redondea a la decena. Su tabla:

id_articulo	nombre	categoria	precio
1	cuaderno universitario 100 hojas	cuaderno	1490
2	cuaderno college 80 hojas	cuaderno	1190
3	lápiz grafito caja 12	escritura	2350
4	goma de borrar	escritura	390
5	cuaderno croquis	cuaderno	2190
6	regla 30 cm	geometría	650

Escribe el `UPDATE` con `ROUND`, di cuántas filas debe cambiar y calcula los precios nuevos a mano. ¿Cuántas habría cambiado sin el `WHERE`? ¿Queda el precio como entero?

8.7 · Borrar un club (intermedio). Un liceo tiene los clubes 1 (ajedrez) y 2 (robótica) y cinco inscritos: Amanda y Damián en ajedrez; Bruno, Catalina y Elena en robótica. El club de robótica se cierra. Escribe la tabla `inscrito` tres veces, con la clave foránea sin acción, con `ON DELETE CASCADE` y con `ON DELETE SET NULL`, y di qué deja cada una después de `DELETE FROM club WHERE id_club = 2;`. ¿Cuál elegirías para el liceo?

8.8 · El número del ticket (intermedio). La mesa de ayuda de una municipalidad numera sus tickets con la base. Se crean tres tickets, se borra el 3 (era un duplicado) y se crea uno nuevo. ¿Qué número recibe con `INTEGER PRIMARY KEY` y cuál con `INTEGER PRIMARY KEY AUTOINCREMENT`? ¿Cuál conviene y cómo se escribe en MySQL?

8.9 · Seis errores en un script (desafío). Este script de un centro médico de Concepción tiene seis errores. Para cada uno di qué mensaje da SQLite (o por qué no da ninguno), qué haría MySQL y cómo se corrige. Escribe el script corregido y comprueba que la hora quede registrada.

```
PRAGMA foreign_keys = ON;
CREATE TABLE medico (
    id_medico  INTEGER PRIMARY KEY,
    nombre     VARCHAR(40) NOT NULL,
    fono       INTEGER
);
CREATE TABLE hora (
    id_hora   INTEGER PRIMARY KEY,
    fecha     DATE NOT NULL,
    id_medico INTEGER NOT NULL,
    FOREIGN KEY id_medico REFERENCES medico(id_medico)
);
ALTER TABLE medico MODIFY COLUMN fono telefono VARCHAR(15);
INSERT INTO medico VALUES (1, Pedro Alarcón, 412345678);
INSERT INTO hora VALUES (2, '05/11/2026', 1);
UPDATE medico SET nombre = 'Pedro Alarcón Rojas' WHERE id_doctor = 1;
```

8.10 · El `UPDATE` sin `WHERE` (desafío). La librería del ejercicio 8.6 sacó en la mañana un respaldo de su tabla (`CREATE TABLE articulo_respaldo AS SELECT * FROM articulo;`); después subió el lápiz grafito a $2.490. En la tarde, el encargado quiso dejar la goma en $450 y escribió `UPDATE articulo SET precio = 450;`. (a) ¿Cuántas filas cambió? (b) Restaura los precios desde el respaldo sin perder el cambio del lápiz y deja la goma en $450. (c) Escribe cómo debió hacerlo para que el error se pudiera deshacer.

---

13. Pautas

8.1. La clave se numera sola por ser `INTEGER PRIMARY KEY`; la fecha y el peso no llevan `NOT NULL` porque pueden no saberse; el `CHECK` del peso deja pasar el `NULL` (una condición desconocida no se rechaza) y rechaza el cero:

```
PRAGMA foreign_keys = ON;
CREATE TABLE mascota (
    id_mascota INTEGER PRIMARY KEY,
    nombre     VARCHAR(30) NOT NULL,
    especie    VARCHAR(10) NOT NULL CHECK (especie IN ('perro', 'gato', 'otro')),
    fecha_nac  DATE,
    peso_kg    DECIMAL(5,1) CHECK (peso_kg > 0),
    rut_dueno  VARCHAR(12) NOT NULL
);
INSERT INTO mascota (nombre, especie, fecha_nac, peso_kg, rut_dueno)
VALUES ('Canela', 'perro', '2019-05-14', 12.5, '42.224.354-2');
SELECT * FROM mascota;
```

id_mascota	nombre	especie	fecha_nac	peso_kg	rut_dueno
1	Canela	perro	2019-05-14	12.5	42.224.354-2

En SQL el decimal se escribe con punto (`12.5`). Las dos filas malas:

```
INSERT INTO mascota (nombre, especie, rut_dueno) VALUES ('Pirata', 'loro', '42.224.354-2');
```

```
CHECK constraint failed: especie IN ('perro', 'gato', 'otro')
```

```
INSERT INTO mascota (nombre, especie, peso_kg, rut_dueno) VALUES ('Mota', 'gato', 0,
'42.224.354-2');
```

```
CHECK constraint failed: peso_kg > 0
```

En MySQL: `id_mascota INT AUTO_INCREMENT PRIMARY KEY`, `INT` en vez de `INTEGER`, y `DECIMAL(5,1)` guarda el 12,5 exacto. Si el consultorio tiene una tabla de dueños, `rut_dueno` debería ser su clave foránea (con la línea `FOREIGN KEY` aparte, para MySQL 8).

8.2. (a) Precio: `INTEGER` / `INT`. Teléfono: `VARCHAR(15)` en ambos. Fecha: `DATE` (en SQLite, texto `'AAAA-MM-DD'`). Peso: `DECIMAL(5,1)`. Esterilizada: `INTEGER` con 0 o 1 en SQLite, `BOOLEAN` en MySQL. Observaciones: `TEXT`. (b) SQLite guarda el texto entero, de 10 letras, en una columna declarada de 5:

```
CREATE TABLE prueba (comuna VARCHAR(5));
INSERT INTO prueba VALUES ('Concepción');
SELECT comuna, length(comuna) FROM prueba;
```

comuna	length(comuna)
Concepción	10

MySQL, en su modo estricto de fábrica, rechaza la fila con el error *Data too long for column*. Un script probado solo en SQLite puede fallar en MySQL por esto. (c) Ordenadas como texto con barras, las fechas quedan en orden de día, no de calendario:

```
CREATE TABLE control (fecha_txt VARCHAR(10), fecha_iso DATE);
INSERT INTO control VALUES ('07/03/2026', '2026-03-07'), ('15/01/2026', '2026-01-15'),
('28/02/2025', '2025-02-28');
SELECT fecha_txt FROM control ORDER BY fecha_txt;
```

fecha_txt
07/03/2026
15/01/2026
28/02/2025

```
SELECT fecha_iso FROM control ORDER BY fecha_iso;
```

fecha_iso
2025-02-28
2026-01-15
2026-03-07

Solo el formato año-mes-día ordena bien como texto, porque compara primero lo más importante.

8.3. Raúl recibe los tres valores por defecto. Gloria queda con la cuota en `NULL`: su `INSERT` nombró la columna y le pasó `NULL`, y `DEFAULT` actúa solo cuando la columna se omite. Sergio recibe la cuota por defecto y conserva su categoría y su `activo`:

id_socio	nombre	categoria	cuota	activo
1	Antonia Bravo	juvenil	9800	1
2	Raúl Espinoza	adulto	14500	1
3	Gloria Pinto	adulto	NULL	1
4	Sergio Vidal	senior	14500	0

Si la cuota nunca debe quedar vacía, falta un `NOT NULL` en `cuota`: con él, el `INSERT` de Gloria habría fallado en vez de guardar un socio sin cuota.

8.4. (a) La tabla padre va primero:

```
PRAGMA foreign_keys = ON;
CREATE TABLE box (
    id_box  INTEGER PRIMARY KEY,
    piso    INTEGER NOT NULL
);
CREATE TABLE atencion (
    id_atencion INTEGER PRIMARY KEY,
    fecha       DATE NOT NULL,
    id_box      INTEGER NOT NULL,
    FOREIGN KEY (id_box) REFERENCES box(id_box)
);
```

(b) La atención apunta a un box que todavía no existe:

```
INSERT INTO atencion VALUES (1, '2026-11-05', 3);
```

`FOREIGN KEY constraint failed`. En el orden correcto entra todo:

```
INSERT INTO box VALUES (3, 1), (7, 2);
INSERT INTO atencion VALUES (1, '2026-11-05', 3), (2, '2026-11-05', 7), (3, '2026-11-06',
3);
SELECT COUNT(*) FROM atencion;
```

COUNT(*)
3

(c) No: borrar `box` dejaría tres atenciones apuntando a la nada.

```
DROP TABLE box;
```

`FOREIGN KEY constraint failed`. Se borra primero el hijo y después el padre:

```
DROP TABLE atencion;
DROP TABLE box;
SELECT COUNT(*) FROM sqlite_master WHERE type = 'table';
```

COUNT(*)
0

8.5. (a) y (b):

```
CREATE TABLE puesto (
    id_puesto INTEGER PRIMARY KEY,
    titular   VARCHAR(40) NOT NULL,
    metros    INTEGER NOT NULL
);
INSERT INTO puesto VALUES (1, 'Olga Retamal', 6), (2, 'Juan Saavedra', 9), (3, 'Elsa
Fuentealba', 4);
ALTER TABLE puesto ADD COLUMN rubro VARCHAR(20) NOT NULL DEFAULT 'verduras';
ALTER TABLE puesto RENAME COLUMN metros TO metros2;
SELECT * FROM puesto;
```

id_puesto	titular	metros2	rubro
1	Olga Retamal	6	verduras
2	Juan Saavedra	9	verduras
3	Elsa Fuentealba	4	verduras

(c) SQLite no tiene `MODIFY`:

```
ALTER TABLE puesto MODIFY COLUMN metros2 DECIMAL(4,1);
```

`near "MODIFY": syntax error`. La reconstrucción: tabla nueva, copia, borrar la vieja, renombrar.

```
CREATE TABLE puesto_nueva (
    id_puesto INTEGER PRIMARY KEY,
    titular   VARCHAR(40) NOT NULL,
    metros2   DECIMAL(4,1) NOT NULL CHECK (metros2 > 0),
    rubro     VARCHAR(20) NOT NULL DEFAULT 'verduras'
);
INSERT INTO puesto_nueva (id_puesto, titular, metros2, rubro)
SELECT id_puesto, titular, metros2, rubro FROM puesto;
DROP TABLE puesto;
ALTER TABLE puesto_nueva RENAME TO puesto;
UPDATE puesto SET metros2 = 4.5 WHERE id_puesto = 3;
SELECT * FROM puesto;
```

id_puesto	titular	metros2	rubro
1	Olga Retamal	6	verduras
2	Juan Saavedra	9	verduras
3	Elsa Fuentealba	4.5	verduras

En MySQL basta `ALTER TABLE puesto MODIFY COLUMN metros2 DECIMAL(4,1) NOT NULL;`. (d):

```
ALTER TABLE puesto DROP COLUMN rubro;
SELECT * FROM puesto;
```

id_puesto	titular	metros2
1	Olga Retamal	6
2	Juan Saavedra	9
3	Elsa Fuentealba	4.5

8.6. La tabla de la librería:

```
CREATE TABLE articulo (
    id_articulo INTEGER PRIMARY KEY,
    nombre      VARCHAR(40) NOT NULL,
    categoria   VARCHAR(20) NOT NULL,
    precio      INTEGER NOT NULL CHECK (precio > 0)
);
INSERT INTO articulo VALUES
    (1, 'cuaderno universitario 100 hojas', 'cuaderno', 1490),
    (2, 'cuaderno college 80 hojas', 'cuaderno', 1190),
    (3, 'lápiz grafito caja 12', 'escritura', 2350),
    (4, 'goma de borrar', 'escritura', 390),
    (5, 'cuaderno croquis', 'cuaderno', 2190),
    (6, 'regla 30 cm', 'geometría', 650);
```

Dividir por 10, redondear y volver a multiplicar por 10 redondea a la decena:

```
UPDATE articulo
SET precio = ROUND(precio * 1.08 / 10) * 10
WHERE categoria = 'cuaderno';
SELECT changes();
```

changes()
3

A mano: $1.490 por 1,08 da 1.609,2, que redondeado a la decena es $1.610; $1.190 da 1.285,2, que queda en $1.290; y $2.190 da 2.365,2, que queda en $2.370. La tabla queda:

```
SELECT * FROM articulo;
```

id_articulo	nombre	categoria	precio
1	cuaderno universitario 100 hojas	cuaderno	1610
2	cuaderno college 80 hojas	cuaderno	1290
3	lápiz grafito caja 12	escritura	2350
4	goma de borrar	escritura	390
5	cuaderno croquis	cuaderno	2370
6	regla 30 cm	geometría	650

Sin el `WHERE` habría subido los 6 artículos. `ROUND` devuelve un número con decimales, pero la columna es entera y SQLite guarda el 1610.0 como el entero 1610:

```
SELECT typeof(precio) FROM articulo WHERE id_articulo = 1;
```

typeof(precio)
integer

8.7. Sin acción en la clave foránea:

```
PRAGMA foreign_keys = ON;
CREATE TABLE club (id_club INTEGER PRIMARY KEY, nombre VARCHAR(30) NOT NULL);
CREATE TABLE inscrito (
    id_inscrito INTEGER PRIMARY KEY,
    nombre      VARCHAR(40) NOT NULL,
    id_club     INTEGER,
    FOREIGN KEY (id_club) REFERENCES club(id_club)
);
INSERT INTO club VALUES (1, 'ajedrez'), (2, 'robótica');
INSERT INTO inscrito VALUES (1, 'Amanda', 1), (2, 'Bruno', 2), (3, 'Catalina', 2), (4,
'Damián', 1), (5, 'Elena', 2);
```

el borrado se rechaza:

```
DELETE FROM club WHERE id_club = 2;
```

`FOREIGN KEY constraint failed`. Con `ON DELETE CASCADE` se van también Bruno, Catalina y Elena:

```
PRAGMA foreign_keys = ON;
CREATE TABLE club (id_club INTEGER PRIMARY KEY, nombre VARCHAR(30) NOT NULL);
CREATE TABLE inscrito (
    id_inscrito INTEGER PRIMARY KEY,
    nombre      VARCHAR(40) NOT NULL,
    id_club     INTEGER,
    FOREIGN KEY (id_club) REFERENCES club(id_club) ON DELETE CASCADE
);
INSERT INTO club VALUES (1, 'ajedrez'), (2, 'robótica');
INSERT INTO inscrito VALUES (1, 'Amanda', 1), (2, 'Bruno', 2), (3, 'Catalina', 2), (4,
'Damián', 1), (5, 'Elena', 2);
DELETE FROM club WHERE id_club = 2;
SELECT * FROM inscrito;
```

id_inscrito	nombre	id_club
1	Amanda	1
4	Damián	1

Con `ON DELETE SET NULL` los tres siguen, sin club:

```
PRAGMA foreign_keys = ON;
CREATE TABLE club (id_club INTEGER PRIMARY KEY, nombre VARCHAR(30) NOT NULL);
CREATE TABLE inscrito (
    id_inscrito INTEGER PRIMARY KEY,
    nombre      VARCHAR(40) NOT NULL,
    id_club     INTEGER,
    FOREIGN KEY (id_club) REFERENCES club(id_club) ON DELETE SET NULL
);
INSERT INTO club VALUES (1, 'ajedrez'), (2, 'robótica');
INSERT INTO inscrito VALUES (1, 'Amanda', 1), (2, 'Bruno', 2), (3, 'Catalina', 2), (4,
'Damián', 1), (5, 'Elena', 2);
DELETE FROM club WHERE id_club = 2;
SELECT * FROM inscrito;
```

id_inscrito	nombre	id_club
1	Amanda	1
2	Bruno	NULL
3	Catalina	NULL
4	Damián	1
5	Elena	NULL

Para el liceo, `SET NULL`: los alumnos siguen siendo alumnos aunque cierre su club, y el liceo puede ofrecerles otro. `CASCADE` borraría a tres estudiantes del registro por cerrar un taller. `SET NULL` exige que `id_club` acepte `NULL` (no lleva `NOT NULL`).

8.8. Con `INTEGER PRIMARY KEY`, el ticket nuevo recibe el mayor valor más uno, y como el mayor (el 3) se borró, el número 3 vuelve a usarse:

```
CREATE TABLE ticket (id_ticket INTEGER PRIMARY KEY, detalle VARCHAR(40) NOT NULL);
INSERT INTO ticket (detalle) VALUES ('impresora sin tóner'), ('wifi del segundo piso'),
('clave bloqueada');
DELETE FROM ticket WHERE id_ticket = 3;
INSERT INTO ticket (detalle) VALUES ('monitor parpadea');
SELECT * FROM ticket;
```

id_ticket	detalle
1	impresora sin tóner
2	wifi del segundo piso
3	monitor parpadea

Con `AUTOINCREMENT`, SQLite recuerda el mayor número entregado y sigue en el 4:

```
CREATE TABLE ticket (id_ticket INTEGER PRIMARY KEY AUTOINCREMENT, detalle VARCHAR(40) NOT
NULL);
INSERT INTO ticket (detalle) VALUES ('impresora sin tóner'), ('wifi del segundo piso'),
('clave bloqueada');
DELETE FROM ticket WHERE id_ticket = 3;
INSERT INTO ticket (detalle) VALUES ('monitor parpadea');
SELECT * FROM ticket;
```

id_ticket	detalle
1	impresora sin tóner
2	wifi del segundo piso
4	monitor parpadea

Conviene `AUTOINCREMENT`: un funcionario que anotó «ticket 3» no debería encontrarse con otro problema bajo ese número. En MySQL: `id_ticket INT AUTO_INCREMENT PRIMARY KEY`, que tampoco reutiliza números.

8.9. Los mensajes de SQLite, uno por uno (cada línea se corre después de corregir las anteriores):

Línea	SQLite dice	Arreglo
`fono INTEGER`	nada	`telefono VARCHAR(15)`: `041 234 5678` como número pierde el 0 y no admite `+56`
`FOREIGN KEY id_medico REFERENCES ...`	`near "id_medico": syntax error`	`FOREIGN KEY (id_medico) REFERENCES medico(id_medico)`
`MODIFY COLUMN fono telefono VARCHAR(15)`	`near "MODIFY": syntax error`	SQLite: `RENAME COLUMN` (o crear bien la tabla); MySQL: `CHANGE COLUMN fono telefono VARCHAR(15)`, porque `MODIFY` no cambia nombres
`VALUES (1, Pedro Alarcón, ...)`	`near "Alarcón": syntax error`	`'Pedro Alarcón'`, entre comillas simples
`'05/11/2026'`	nada: lo guarda como texto	`'2026-11-05'`; MySQL lo rechaza como fecha incorrecta
`WHERE id_doctor = 1`	`no such column: id_doctor`	`WHERE id_medico = 1`

Los dos errores que no dan mensaje en SQLite (el teléfono numérico y la fecha con barras) son los peores: el script «funciona» y los datos quedan mal. Las líneas que fallan, tal cual:

```
CREATE TABLE hora (
    id_hora   INTEGER PRIMARY KEY,
    fecha     DATE NOT NULL,
    id_medico INTEGER NOT NULL,
    FOREIGN KEY id_medico REFERENCES medico(id_medico)
);
```

```
ALTER TABLE medico MODIFY COLUMN fono telefono VARCHAR(15);
```

```
INSERT INTO medico VALUES (1, Pedro Alarcón, 412345678);
```

```
UPDATE medico SET nombre = 'Pedro Alarcón Rojas' WHERE id_doctor = 1;
```

El script corregido:

```
DROP TABLE medico;
CREATE TABLE medico (
    id_medico  INTEGER PRIMARY KEY,
    nombre     VARCHAR(40) NOT NULL,
    telefono   VARCHAR(15)
);
CREATE TABLE hora (
    id_hora   INTEGER PRIMARY KEY,
    fecha     DATE NOT NULL,
    id_medico INTEGER NOT NULL,
    FOREIGN KEY (id_medico) REFERENCES medico(id_medico)
);
INSERT INTO medico VALUES (1, 'Pedro Alarcón', '+56 41 234 5678');
INSERT INTO hora VALUES (1, '2026-11-05', 1);
UPDATE medico SET nombre = 'Pedro Alarcón Rojas' WHERE id_medico = 1;
SELECT h.id_hora, h.fecha, m.nombre, m.telefono FROM hora h JOIN medico m ON m.id_medico =
h.id_medico;
```

id_hora	fecha	nombre	telefono
1	2026-11-05	Pedro Alarcón Rojas	+56 41 234 5678

La prueba de que la fecha con barras no es una fecha: la función `date` de SQLite la convierte en `NULL`.

```
INSERT INTO hora VALUES (2, '05/11/2026', 1);
SELECT id_hora, fecha, date(fecha) FROM hora;
```

id_hora	fecha	date(fecha)
1	2026-11-05	2026-11-05
2	05/11/2026	NULL

8.10. La tabla de la tarde, antes del error, con el respaldo de la mañana y el cambio del lápiz:

```
CREATE TABLE articulo (
    id_articulo INTEGER PRIMARY KEY,
    nombre      VARCHAR(40) NOT NULL,
    categoria   VARCHAR(20) NOT NULL,
    precio      INTEGER NOT NULL CHECK (precio > 0)
);
INSERT INTO articulo VALUES
    (1, 'cuaderno universitario 100 hojas', 'cuaderno', 1610),
    (2, 'cuaderno college 80 hojas', 'cuaderno', 1290),
    (3, 'lápiz grafito caja 12', 'escritura', 2350),
    (4, 'goma de borrar', 'escritura', 390),
    (5, 'cuaderno croquis', 'cuaderno', 2370),
    (6, 'regla 30 cm', 'geometría', 650);
CREATE TABLE articulo_respaldo AS SELECT * FROM articulo;
UPDATE articulo SET precio = 2490 WHERE id_articulo = 3;
```

(a) El error cambió las 6 filas:

```
UPDATE articulo SET precio = 450;
SELECT changes();
```

changes()
6

(b) Un `UPDATE` con una subconsulta trae, para cada artículo, su precio del respaldo; el lápiz se excluye porque su precio bueno es posterior al respaldo, y se vuelve a escribir. Al final va el cambio que se quería hacer:

```
UPDATE articulo
SET precio = (SELECT r.precio FROM articulo_respaldo r WHERE r.id_articulo =
articulo.id_articulo)
WHERE id_articulo <> 3;
UPDATE articulo SET precio = 2490 WHERE id_articulo = 3;
UPDATE articulo SET precio = 450 WHERE id_articulo = 4;
SELECT * FROM articulo;
```

id_articulo	nombre	categoria	precio
1	cuaderno universitario 100 hojas	cuaderno	1610
2	cuaderno college 80 hojas	cuaderno	1290
3	lápiz grafito caja 12	escritura	2490
4	goma de borrar	escritura	450
5	cuaderno croquis	cuaderno	2370
6	regla 30 cm	geometría	650

Sin el respaldo no habría forma de saber los precios anteriores: `UPDATE` no guarda historia. (c) Dentro de una transacción, mirando `changes()` antes de confirmar:

```
BEGIN;
UPDATE articulo SET precio = 450;
SELECT changes();
```

changes()
6

Seis filas cuando se esperaba una: se deshace.

```
ROLLBACK;
SELECT precio FROM articulo WHERE id_articulo = 1;
```

precio
1610

Y la instrucción correcta era `UPDATE articulo SET precio = 450 WHERE id_articulo = 4;`, que en MySQL con `sql_safe_updates` es la única de las dos que se habría aceptado.

---

14. Caso integrador · Estampa Sur escribe su base, carga abril y corrige precios

> Este caso sigue a Estampa Sur por todo el ramo. Las unidades 4 y 5 diseñaron sus 8 tablas y la unidad 7 les hizo las primeras preguntas. Acá se lee el `CREATE TABLE` de cada una, se carga la primera semana de abril y se corrigen precios. La pieza modifica datos: se corre sobre una **copia** de la base, nunca sobre `datos/estampa_sur.db`.

El esquema, tabla por tabla

`datos/esquema.sql` es el script que crea la base. Lo escribe el script del caso, y las tablas van en el orden en que se pueden crear: primero las que no apuntan a nadie.

```
CREATE TABLE tienda (
    id_tienda   INTEGER PRIMARY KEY,
    nombre      VARCHAR(40) NOT NULL UNIQUE,
    comuna      VARCHAR(40) NOT NULL
);
CREATE TABLE producto (
    id_producto INTEGER PRIMARY KEY,
    nombre      VARCHAR(40) NOT NULL UNIQUE,
    precio      INTEGER NOT NULL CHECK (precio > 0),
    costo_var   INTEGER NOT NULL CHECK (costo_var > 0),
    seg_estampado INTEGER NOT NULL
);
CREATE TABLE proveedor (
    id_proveedor INTEGER PRIMARY KEY,
    nombre       VARCHAR(40) NOT NULL,
    rubro        VARCHAR(30) NOT NULL
);
```

tienda, producto y proveedor son las tablas maestras. Las tres tienen una clave que se numera sola. El nombre de una tienda y el de un producto no se repiten (`UNIQUE`): sería imposible saber de cuál se habla. El de un proveedor sí podría repetirse (dos empresas pueden llamarse igual); lo que los distingue es la clave. Precio y costo variable de un producto son pesos enteros y positivos (`CHECK`); `seg_estampado` son los segundos de estampado por unidad, que usan los ramos de operaciones.

```
CREATE TABLE suministra (
    id_proveedor INTEGER NOT NULL REFERENCES proveedor(id_proveedor),
    id_producto  INTEGER NOT NULL REFERENCES producto(id_producto),
    costo_unidad INTEGER NOT NULL,
    PRIMARY KEY (id_proveedor, id_producto)
);
```

suministra es la relación muchos a muchos entre proveedores y productos: su clave es el par, y el costo es un atributo del par (lo que cobra *ese* proveedor por *ese* producto). Las dos columnas de la clave llevan `NOT NULL` explícito por la rareza de SQLite de la sección 3.

```
CREATE TABLE cliente (
    id_cliente  INTEGER PRIMARY KEY,
    nombre      VARCHAR(60) NOT NULL,
    tipo        VARCHAR(12) NOT NULL CHECK (tipo IN ('colegio', 'club', 'empresa',
'persona')),
    id_tienda   INTEGER NOT NULL REFERENCES tienda(id_tienda)
);
CREATE TABLE pedido (
    id_pedido   INTEGER PRIMARY KEY,
    id_cliente  INTEGER NOT NULL REFERENCES cliente(id_cliente),
    fecha       DATE NOT NULL,
    estado      VARCHAR(12) NOT NULL CHECK (estado IN ('pendiente', 'en taller',
'entregado'))
);
```

cliente tiene un tipo cerrado (el `CHECK ... IN` hace de lista de valores permitidos) y pertenece a una tienda. **pedido** es de un cliente, con fecha y un estado también cerrado: un `'despachado'` escrito por error no entra.

```
CREATE TABLE detalle_pedido (
    id_pedido   INTEGER NOT NULL REFERENCES pedido(id_pedido),
    id_producto INTEGER NOT NULL REFERENCES producto(id_producto),
    cantidad    INTEGER NOT NULL CHECK (cantidad > 0),
    precio_unit INTEGER NOT NULL,
    PRIMARY KEY (id_pedido, id_producto)
);
CREATE TABLE orden_produccion (
    id_orden    INTEGER PRIMARY KEY,
    id_producto INTEGER NOT NULL REFERENCES producto(id_producto),
    semana      INTEGER NOT NULL,
    lote        INTEGER NOT NULL
);
```

detalle_pedido es la tabla que la unidad 6 sacó del pedido al normalizar: una línea por producto pedido, con clave compuesta (un producto aparece una vez por pedido; si se piden más, sube la cantidad). Guarda su propio `precio_unit`, el precio **del día del pedido**: si mañana cambia la lista, los pedidos viejos no cambian (se comprueba abajo). **orden_produccion** guarda los lotes que el plan maestro de producción manda a fabricar cada semana.

En total, 7 claves foráneas, todas escritas al lado de la columna.

Para MySQL, ese estilo no sirve en las versiones 8.0 y 8.4: las siete se ignorarían sin aviso. `sql/u08_esquema_mysql.sql` trae el mismo esquema con las claves foráneas en líneas `FOREIGN KEY` aparte, `AUTO_INCREMENT` en las claves que se numeran solas y `ENGINE=InnoDB`. Por ejemplo:

```
CREATE TABLE detalle_pedido (
    id_pedido   INT NOT NULL,
    id_producto INT NOT NULL,
    cantidad    INT NOT NULL CHECK (cantidad > 0),
    precio_unit INT NOT NULL,
    PRIMARY KEY (id_pedido, id_producto),
    FOREIGN KEY (id_pedido) REFERENCES pedido(id_pedido),
    FOREIGN KEY (id_producto) REFERENCES producto(id_producto)
) ENGINE=InnoDB;
```

El verificador carga esa versión en SQLite (quitando lo que es solo de MySQL) y comprueba que tenga las mismas columnas, claves primarias y claves foráneas que `datos/esquema.sql`.

La primera semana de abril

Todo lo que sigue está en `sql/u08_caso.sql`. Primero, la copia y la llave de las claves foráneas:

```
copy datos\estampa_sur.db datos\estampa_sur_copia.db        (en macOS o Linux: cp)
sqlite3 datos\estampa_sur_copia.db
```

```
PRAGMA foreign_keys = ON;
SELECT COUNT(*) FROM pedido;
```

COUNT(*)
120

Los 120 pedidos de marzo. Entre el 01-04 y el 07-04-2026 llegan seis pedidos nuevos. Dos son de clientes que nunca habían comprado (el 27 y el 29):

```
INSERT INTO pedido (id_pedido, id_cliente, fecha, estado) VALUES
    (121, 27, '2026-04-01', 'pendiente'),
    (122,  3, '2026-04-01', 'en taller'),
    (123, 12, '2026-04-02', 'en taller'),
    (124, 29, '2026-04-03', 'pendiente'),
    (125,  8, '2026-04-06', 'pendiente'),
    (126, 21, '2026-04-07', 'pendiente');
SELECT changes();
```

changes()
6

Y sus 9 líneas, en una sola instrucción. Los pedidos van antes que las líneas, porque las líneas apuntan a ellos:

```
INSERT INTO detalle_pedido (id_pedido, id_producto, cantidad, precio_unit) VALUES
    (121, 1,  4, 3990),
    (121, 5,  2, 3190),
    (122, 2, 35, 9890),
    (123, 3, 60, 6290),
    (123, 4, 60, 2490),
    (124, 2,  3, 9980),
    (125, 4, 25, 2490),
    (126, 1,  2, 3990),
    (126, 5,  1, 3190);
SELECT COUNT(*) FROM detalle_pedido;
```

COUNT(*)
221

De 212 líneas a 221. **Cuatro digitaciones que la base rechaza:** un pedido de un cliente que no existe, un estado inventado, una línea con cantidad cero y una segunda línea del mismo producto en el mismo pedido.

```
INSERT INTO pedido VALUES (127, 31, '2026-04-07', 'pendiente');
```

```
FOREIGN KEY constraint failed
```

```
INSERT INTO pedido VALUES (127, 5, '2026-04-07', 'despachado');
```

```
CHECK constraint failed: estado IN ('pendiente', 'en taller', 'entregado')
```

```
INSERT INTO detalle_pedido VALUES (126, 3, 0, 6290);
```

```
CHECK constraint failed: cantidad > 0
```

```
INSERT INTO detalle_pedido VALUES (126, 1, 3, 3990);
```

```
UNIQUE constraint failed: detalle_pedido.id_pedido, detalle_pedido.id_producto
```

Y lo que pasa sin la llave. Con las claves foráneas apagadas, el pedido del cliente 31 entra sin reclamo:

```
PRAGMA foreign_keys = OFF;
INSERT INTO pedido VALUES (127, 31, '2026-04-07', 'pendiente');
SELECT changes();
```

changes()
1

Un pedido huérfano: nadie podría saber de qué tienda es ni a quién cobrárselo. Se borra y se vuelve a encender la llave:

```
DELETE FROM pedido WHERE id_pedido = 127;
PRAGMA foreign_keys = ON;
SELECT changes();
```

changes()
1

Dos correcciones de precio

Una línea mal digitada. Al revisar la semana, el precio de los polerones del pedido 124 no calza con la lista:

```
SELECT d.id_pedido, pr.nombre, d.cantidad, d.precio_unit, pr.precio
FROM detalle_pedido d JOIN producto pr ON pr.id_producto = d.id_producto
WHERE d.id_pedido > 120
ORDER BY d.id_pedido, d.id_producto;
```

id_pedido	nombre	cantidad	precio_unit	precio
121	poleras básicas	4	3990	3990
121	gorros	2	3190	3190
122	polerones	35	9890	9890
123	poleras estampado doble	60	6290	6290
123	bolsas	60	2490	2490
124	polerones	3	9980	9890
125	bolsas	25	2490	2490
126	poleras básicas	2	3990	3990
126	gorros	1	3190	3190

Se digitó 9.980 en vez de 9.890. La clave de `detalle_pedido` es compuesta, así que el `WHERE` lleva las dos columnas; con solo `id_producto = 2` se habrían cambiado los polerones de todos los pedidos.

```
UPDATE detalle_pedido
SET precio_unit = 9890
WHERE id_pedido = 124 AND id_producto = 2;
SELECT changes();
```

changes()
1

La semana suma 192 unidades en 9 líneas. Con el error, las ventas daban $998.650; corregidas son $998.380 (el cliente 29 se iba a cobrar $270 de más):

```
SELECT SUM(d.cantidad * d.precio_unit)
FROM detalle_pedido d JOIN pedido p ON p.id_pedido = d.id_pedido
WHERE p.fecha BETWEEN '2026-04-01' AND '2026-04-07';
```

SUM(d.cantidad * d.precio_unit)
998380

Un cambio de lista. Desde el 08-04 los gorros suben de $3.190 a $3.390. ¿Cambian las ventas de marzo? Antes:

```
SELECT SUM(d.cantidad * d.precio_unit)
FROM detalle_pedido d JOIN pedido p ON p.id_pedido = d.id_pedido
WHERE p.fecha < '2026-04-01';
```

SUM(d.cantidad * d.precio_unit)
35723440

```
UPDATE producto
SET precio = 3390
WHERE id_producto = 5;
SELECT changes();
```

changes()
1

Y después:

```
SELECT SUM(d.cantidad * d.precio_unit)
FROM detalle_pedido d JOIN pedido p ON p.id_pedido = d.id_pedido
WHERE p.fecha < '2026-04-01';
```

SUM(d.cantidad * d.precio_unit)
35723440

Las ventas de marzo siguen en $35.723.440: cada línea guardó su precio del día. Si `detalle_pedido` leyera el precio desde `producto`, este `UPDATE` habría reescrito la historia de marzo. Es la misma cifra que suma la unidad 10 al agrupar las ventas por tienda.

El mismo cambio, sin `WHERE`. Para ver el daño sin sufrirlo, dentro de una transacción:

```
BEGIN;
UPDATE producto SET precio = 3390;
SELECT changes();
```

changes()
5

Cinco productos a $3.390, incluidos los polerones de $9.890. Se deshace:

```
ROLLBACK;
SELECT id_producto, nombre, precio FROM producto;
```

id_producto	nombre	precio
1	poleras básicas	3990
2	polerones	9890
3	poleras estampado doble	6290
4	bolsas	2490
5	gorros	3390

Solo los gorros quedan con el precio nuevo, que se había confirmado antes del `BEGIN`.

Y un efecto de la carga:

```
SELECT COUNT(*) FROM cliente
WHERE id_cliente NOT IN (SELECT id_cliente FROM pedido);
```

COUNT(*)
2

Quedan 2 clientes sin pedidos (el 28 y el 30), de los 4 que encontró la unidad 7.

La cadena. El verificador arma la copia en memoria con los datos del script del caso y corre `sql/u08_caso.sql` entero. Comprueba que `datos/esquema.sql` sea exactamente el esquema del script, que las tablas partan con los conteos del caso (120 pedidos, 212 líneas), que las ventas de marzo ($35.723.440) sean la suma de las ventas por tienda del caso y que los clientes sin pedidos bajen de 4 a 2 (`CADENA: OK`).

Lo que pasa a las unidades siguientes. La unidad 9 consulta estas tablas de a una (filtrar, ordenar, contar), la 10 las junta y agrega, y la 12 vuelve a registrar un pedido como una sola transacción: si falla una línea, no queda un pedido a medias.

---

15. Para seguir

En los libros del área. El programa del ramo no fija bibliografía. Elmasri y Navathe, *Fundamentos de sistemas de bases de datos*, y Silberschatz, Korth y Sudarshan, *Fundamentos de bases de datos*, traen en sus capítulos de SQL la definición de datos, las restricciones y la modificación de la base con la notación de esta unidad.

Abiertos y gratis.

- Adrienne Watt, *Database Design*, 2.ª ed. (BCcampus, licencia CC BY 4.0), capítulos 15 «SQL Structured Query Language» y 16 «SQL Data Manipulation Language», en inglés. https://opentextbc.ca/dbdesign01/
- La documentación de SQLite explica las afinidades de tipo (https://www.sqlite.org/datatype3.html), el `CREATE TABLE` (https://www.sqlite.org/lang_createtable.html) y las claves foráneas (https://www.sqlite.org/foreignkeys.html).

- *MySQL 8.4 Reference Manual*, sección `CREATE TABLE`, con la nota sobre los `REFERENCES` en línea. https://dev.mysql.com/doc/refman/8.4/en/create-table.html

Con el tutor. La carpeta `tutor/` trae el contexto de este ramo. Pégalo al inicio de la conversación con cualquier IA y pídele, por ejemplo: «Dame el enunciado de una base pequeña con cinco reglas de negocio, deja que escriba el `CREATE TABLE` y después pásame seis `INSERT` para que prediga cuáles fallan y con qué mensaje».

Base abierta de ingeniería · Camilo Merino · Bases de Datos · versión 0.1 (16-10-2026). Ejercicios y datos originales; las cifras se verifican con el script de la unidad.